

Real-time Examples of Encapsulation Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Encapsulation Principle in C#**. Please read our [Encapsulation in C#](#) article, where we discussed the basic concepts of the Encapsulation Principle. At the end of this article, you will understand the following Real-time Examples using the Encapsulation Principle in C#.

1. [What is Encapsulation in C#?](#)
2. [Bank Account Management System](#)
3. [Coffee Machine](#)
4. [Car's Speedometer](#)
5. [Digital Wallet](#)
6. [Managing User Permissions in a Software System](#)
7. [Library Book Management System](#)
8. [Advantages and Disadvantages of Encapsulation Principle in C#](#)
9. [When to use the Encapsulation Principle in C#?](#)

What is Encapsulation in C#?

Encapsulation is one of the fundamental principles of Object-Oriented Programming (OOP). In C#, as with other OOP languages, encapsulation refers to bundling related data and behaviors into a single unit (typically a class) and restricting access to some of the object's components. It helps protect the data's integrity and expose only what's necessary to the outside world.

Real-time Example of Encapsulation Principle in C#: Bank Account Management System

One real-time example of encapsulation is a bank account management system. Let's consider the scenario where a bank customer can deposit or withdraw money from their account. Still, certain rules and validations are applied, such as ensuring that the balance does not go below a minimum limit or the customer can't withdraw more than what they have in the account. Let us see how we can implement this example using the Encapsulation Principle in C#:

```
using System;
namespace EncapsulationPrincipleCSharp
{
    public class BankAccount
    {
        // This private field is encapsulated and can't be directly accessed from
        the class.
        private decimal balance;
        public decimal Balance
        {
            // Only provides a way to read the balance but not modify it directly
            get { return balance; }
        }
        public BankAccount(decimal initialBalance)
        {
```

```

        if (initialBalance < 0)
        {
            throw new ArgumentException("Initial balance cannot be negative");
        }
        balance = initialBalance;
    }
    public void Deposit(decimal amount)
    {
        if (amount <= 0)
        {
            throw new ArgumentException("Deposit amount should be positive");
        }
        balance += amount;
    }
    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
        {
            throw new ArgumentException("Withdrawal amount should be positive");
        }
        if (balance - amount < 0)
        {
            throw new InvalidOperationException("Insufficient funds.");
        }
        balance -= amount;
    }
}
//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        // Starts with a balance of 500
        BankAccount myAccount = new BankAccount(500);
        // Balance becomes 700
        myAccount.Deposit(200);
        Console.WriteLine(myAccount.Balance); // Outputs: 700
        // Balance becomes 600
        myAccount.Withdraw(100);
        Console.WriteLine(myAccount.Balance); // Outputs: 600
        // myAccount.balance = -1000; // This would be an error, as the balance
        field is private and inaccessible directly.

        Console.Read();
    }
}
}

```

In this Example:

- The BankAccount class encapsulates the balance field, meaning it can't be directly accessed or modified from outside the class. Instead, operations like depositing or withdrawing are controlled by methods (Deposit and Withdraw).
- The Balance property only provides a getter, ensuring the balance can be viewed but not changed directly.
- Methods Deposit and Withdraw encapsulate the business rules of our banking system, such as ensuring you can't have a negative initial balance, can't deposit negative amounts, and can't withdraw more than you have.

This approach keeps the integrity of the account data, ensuring it can't be misused or mishandled by external forces.

Encapsulation Real-time Example in C#: Coffee Machine

Let's consider another Real-time Example of the coffee machine. A coffee machine has internal mechanisms and operations (like grinding coffee beans, heating water, etc.) that the user shouldn't be concerned about. The user selects the type of coffee they want, and the machine delivers the final product. Let us see how we can implement this example using the Encapsulation Principle in C#:

```
using System;
namespace EncapsulationPrincipleCSharp
{
    public class CoffeeMachine
    {
        private int waterAmount; // in milliliters
        private int beansAmount; // in grams
        private bool isHeated;
        public CoffeeMachine(int water, int beans)
        {
            waterAmount = water;
            beansAmount = beans;
            isHeated = false;
        }
        private void HeatWater()
        {
            if (!isHeated)
            {
                Console.WriteLine("Heating water...");
                isHeated = true;
            }
        }
        private void GrindBeans(int amount)
        {
            if (beansAmount < amount)
            {
                throw new InvalidOperationException("Not enough coffee beans!");
            }
            Console.WriteLine("Grinding coffee beans...");
            beansAmount -= amount;
        }
        // This is the method exposed to the user.
```

```

    public void MakeEspresso()
    {
        HeatWater();
        GrindBeans(20); // let's say we need 20 grams of beans for an espresso
        Console.WriteLine("Making Espresso...");
    }
    // Another method exposed to the user.
    public void MakeLatte()
    {
        HeatWater();
        GrindBeans(25); // we need 25 grams of beans for a latte
        Console.WriteLine("Making Latte...");
    }
    public int BeansLeft()
    {
        return beansAmount;
    }
    public int WaterLeft()
    {
        return waterAmount;
    }
}

//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        CoffeeMachine myMachine = new CoffeeMachine(1000, 100); // Initialize
        1000 ml of water and 100 grams of beans
        myMachine.MakeEspresso(); // Outputs: Heating water... Grinding coffee
        beans... Making Espresso...
        Console.WriteLine($"Beans left: {myMachine.BbeansLeft()} grams");
        Beans left: 80 grams
        Console.Read();
    }
}

```

In the above example:

- The CoffeeMachine class encapsulates the internal workings of the machine. Methods like HeatWater and GrindBeans are private and cannot be accessed outside the class.
- Only methods meant for public use, like MakeEspresso and MakeLatte, are exposed. This ensures the user interacts with the machine in a controlled manner.
- The internal state of the coffee machine (water and bean amounts, whether water is heated or not) is protected from external manipulation, ensuring its integrity.

This is a simplified representation of a coffee machine, but it illustrates the encapsulation concept: hiding the internal complexity and exposing only what's necessary for the user. When you run the above code, you will get the following output:

```
Heating water...
Grinding coffee beans...
Making Espresso...
Beans left: 80 grams
```

Encapsulation Real-time Example in C#: Car's speedometer

Consider another real-world example: a car's speedometer and related components. A car's speedometer displays the current speed to the driver. Internally, this speed is determined by several factors and calculations related to wheel rotations and gear ratios, but the driver only needs to see the final speed value. They don't need to know (and often don't want to know) the complex calculations and mechanisms involved. Let us see how we can implement this example using the Encapsulation Principle in C#:

```
using System;
namespace EncapsulationPrincipleCSharp
{
    public class Car
    {
        private int wheelRotations;
        private double gearRatio = 4.2; // Simplified representation of gear
        private const double WHEEL_CIRCUMFERENCE = 2.0; // Assuming each wheel
covers 2 meters
        // Constructor to initialize the car
        public Car()
        {
            wheelRotations = 0;
        }
        // This method simulates the car moving and the wheel rotating
        private void Move()
        {
            // For simplicity, we're assuming each call to Move represents one
rotation.
            wheelRotations++;
        }
        private double CalculateSpeed()
        {
            // Calculates speed based on wheel rotations and gear ratio
            double distanceCovered = wheelRotations * WHEEL_CIRCUMFERENCE;
            double time = 1.0; // Assume 1 second for simplicity
            return (distanceCovered * gearRatio) / time;
        }
        // Public method to drive the car
        public void Drive()
        {
            Move();
            Console.WriteLine($"Current speed: {CalculateSpeed()} m/s");
        }
        // Let's say there's a mechanism to reset the speedometer
        public void ResetSpeedometer()
    }
}
```

```

        {
            wheelRotations = 0;
        }
    }

//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        Car myCar = new Car();
        myCar.Drive(); // Outputs: Current speed: 8.4 m/s
        myCar.Drive(); // Outputs: Current speed: 16.8 m/s
        Console.Read();
    }
}
}

```

In the above Example:

- The car's internal mechanism (e.g., the Move method and wheelRotations variable) is hidden from the driver.
- The driver only interacts with the Drive and ResetSpeedometer methods, which are exposed for public use.
- The CalculateSpeed method, though internal, takes care of the logic related to determining the current speed based on the wheel rotations and gear ratio.

Here, the complexity of calculating the speed and the internal state management related to wheel rotations are encapsulated within the Car class, and the driver only sees the final speed result.

Encapsulation Principle Real-time Example in C#: Digital Wallet

Let's see a different example to understand the Encapsulation Principle in C#, i.e., a digital wallet. A digital wallet stores money and users can perform operations such as depositing, withdrawing, and checking their balance. Internally, there might be various checks, encryption, and transaction logging, but these details are abstracted from the user. Let us see how we can implement this example using the Encapsulation Principle in C#:

```

using System;
using System.Collections.Generic;
namespace EncapsulationPrincipleCSharp
{
    public class DigitalWallet
    {
        private decimal balance;
        private string walletPassword;
        private List<string> transactionLog = new List<string>();
        public DigitalWallet(string initialPassword)
        {
            balance = 0m;

```

```

        walletPassword = initialPassword;
    }
    private void LogTransaction(string transactionDetail)
    {
        // This can be more complex, involving timestamps and other detail
        transactionLog.Add(transactionDetail);
    }
    public void Deposit(decimal amount)
    {
        if (amount <= 0)
        {
            throw new ArgumentException("Deposit amount should be positive");
        }
        balance += amount;
        LogTransaction($"Deposited {amount}. Current Balance: {balance}");
    }
    public bool Withdraw(decimal amount, string password)
    {
        if (amount <= 0)
        {
            throw new ArgumentException("Withdrawal amount should be positive");
        }
        if (walletPassword != password)
        {
            LogTransaction($"Failed withdrawal attempt of {amount}. Incorrect password.");
            return false;
        }
        if (balance - amount < 0)
        {
            LogTransaction($"Failed withdrawal attempt of {amount}. Insufficient funds.");
            return false;
        }
        balance -= amount;
        LogTransaction($"Withdrew {amount}. Current Balance: {balance}");
        return true;
    }
    public decimal CheckBalance(string password)
    {
        if (walletPassword == password)
        {
            return balance;
        }
        else
        {
            throw new UnauthorizedAccessException("Incorrect password.");
        }
    }
    // This can be used for auditing purposes.
    public List<string> GetTransactionLog(string password)

```

```

        {
            if (walletPassword == password)
            {
                return new List<string>(transactionLog); // Return a copy to
encapsulation.
            }
            else
            {
                throw new UnauthorizedAccessException("Incorrect password.");
            }
        }
    }

//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        DigitalWallet myWallet = new DigitalWallet("securePass123");
        myWallet.Deposit(200m);
        bool isWithdrawn = myWallet.Withdraw(50m, "securePass123"); // Out
        decimal currentBalance = myWallet.CheckBalance("securePass123"); ,
150

        Console.Read();
    }
}
}

```

In this Example:

- The DigitalWallet class encapsulates the balance and the transaction log.
- Deposit and withdrawal methods are designed with safety checks, and the transaction log records all transactions.
- Password verification is required for certain actions, enhancing security.
- GetTransactionLog returns a copy of the transaction log list to prevent external modifications, further emphasizing encapsulation.

The user interacts with clear public methods while the wallet handles the complex internal logic, checks, and record-keeping.

Encapsulation Principle Real-time Example in C#: Managing User Permissions in a Software System

Let's understand encapsulation differently: managing user permissions in a software system. Imagine a system where users have different roles, like "Admin," "User," and "Guest." Each role has different permissions, like "Read," "Write," and "Delete." The system must encapsulate and manage these permissions to ensure security and proper access. Let us see how we can implement this example using the Encapsulation Principle in C#:

```

using System;
using System.Collections.Generic;
namespace EncapsulationPrincipleCSharp
{
    public class User
    {
        private string name;
        private Role userRole;
        public User(string name, Role role)
        {
            this.name = name;
            this.userRole = role;
        }
        public string Name => name;
        public bool HasPermission(PermissionType permission)
        {
            return userRole.HasPermission(permission);
        }
    }
    public enum PermissionType
    {
        Read,
        Write,
        Delete
    }
    public class Role
    {
        private string roleName;
        private List<PermissionType> permissions = new List<PermissionType>();
        public Role(string roleName, List<PermissionType> permissions)
        {
            this.roleName = roleName;
            this.permissions = permissions;
        }
        public bool HasPermission(PermissionType permission)
        {
            return permissions.Contains(permission);
        }
    }
    public class SystemManager
    {
        public void AccessResource(User user, PermissionType permission)
        {
            if (user.HasPermission(permission))
            {
                Console.WriteLine($"{user.Name} has {permission} permission ar
access the resource.");
            }
            else
            {

```

```

        Console.WriteLine($"{user.Name} does not have {permission} per
cannot access the resource.");
    }
}

//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        Role adminRole = new Role("Admin", new List<PermissionType> {
PermissionType.Read, PermissionType.Write, PermissionType.Delete });
        Role userRole = new Role("User", new List<PermissionType> {
PermissionType.Read, PermissionType.Write });
        User alice = new User("Alice", adminRole);
        User bob = new User("Bob", userRole);
        SystemManager manager = new SystemManager();
        manager.AccessResource(alice, PermissionType.Delete); // Outputs:
Delete permission and can access the resource.
        manager.AccessResource(bob, PermissionType.Delete); // Outputs: Bc
have Delete permission and cannot access the resource.
        Console.Read();
    }
}
}

```

In this Example:

- The User class encapsulates its name and associated role. It does not expose how permissions are checked or the details of their role but provides a clear interface (HasPermission) to check permissions.
- The Role class encapsulates a role's name and associated permissions.
- The SystemManager uses encapsulation in the User and Role classes to manage and control resource access.

This design hides the complexity of permission checking, ensures a consistent and secure way of checking permissions, and provides a clear interface for interaction.

Encapsulation Real-time Example in C#: Library Book Management System

Here's another real-world example demonstrating encapsulation using a library book management system. In this example, we have a LibraryBook class. Each book has a title, an ISBN, and a status indicating whether the book is currently checked out. The class should only allow the status to change using designated methods, such as CheckOut and ReturnBook, to ensure the system tracks the book's status correctly. Let us see how we can implement this example using the Encapsulation Principle in C#:

```

using System;
namespace EncapsulationPrincipleCSharp
{

```

```

public class LibraryBook
{
    private string title;
    private string isbn;
    private bool isCheckedOut;
    public LibraryBook(string title, string isbn)
    {
        this.title = title;
        this.isbn = isbn;
        this.isCheckedOut = false;
    }
    public string Title => title;
    public string ISBN => isbn;
    public bool IsCheckedOut => isCheckedOut;
    public void CheckOut()
    {
        if (isCheckedOut)
        {
            Console.WriteLine($"{title} is already checked out.");
        }
        else
        {
            isCheckedOut = true;
            Console.WriteLine($"Successfully checked out {title}.");
        }
    }
    public void ReturnBook()
    {
        if (!isCheckedOut)
        {
            Console.WriteLine($"{title} is not checked out. No need to ret
        }
        else
        {
            isCheckedOut = false;
            Console.WriteLine($"Successfully returned {title}.");
        }
    }
}

//Testing Encapsulation Principle
public class Program
{
    static void Main(string[] args)
    {
        LibraryBook book = new LibraryBook("Moby Dick", "1234567890");
        book.CheckOut(); // Outputs: Successfully checked out Moby Dick.
        book.CheckOut(); // Outputs: Moby Dick is already checked out.
        book.ReturnBook(); // Outputs: Successfully returned Moby Dick.
        Console.Read();
    }
}

```

```
}  
}
```

In this Example:

- The internal state of the LibraryBook (i.e., whether it's checked out) is encapsulated within the class. External entities can't directly modify this state.
- The book's status is changed using specific methods (CheckOut and ReturnBook), which contain logic to ensure the operations are performed correctly.
- Class users can see if the book is checked out via the IsCheckedOut property, but they can't directly change the status.

This encapsulation ensures that the system can reliably track the status of each book and prevent misuse.

Advantages and Disadvantages of Encapsulation Principle in C#

Encapsulation is one of the four foundational principles of Object-Oriented Programming (OOP). It refers to bundling data (attributes) and methods (functions) that operate on the data into a single unit or class and restricting direct access to some of the object's components. Encapsulation is a key principle in C#, like in many other object-oriented languages. Here are its advantages and disadvantages:

Advantages of Encapsulation Principle in C#:

- **Maintainability:** Encapsulation helps in organizing code for maintainability. Since the internal workings are hidden, changes to the logic or data do not affect the external code that uses the object.
- **Flexibility and Extensibility:** You can change the internal implementation of a class without affecting the classes that use it.
- **Control:** By exposing only necessary parts of the object, you can have a high degree of control over the class's functionality and data. For example, you can use validation within a class's set method to enforce specific constraints.
- **Increased Security:** Encapsulation protects the integrity of the data by allowing only controlled and valid operations on it. It prevents unauthorized access and accidental modification of data.
- **Abstraction:** It provides a clear separation between what an object does and how it achieves what it does. This separation allows developers to work on complex systems by understanding the interfaces between objects without knowing the inner workings.
- **Reduction of Side Effects:** By restricting direct access to the object's data, you can ensure that all operations on that data are done through well-defined interfaces, which reduces unforeseen errors or side effects.

Disadvantages of Encapsulation Principle in C#:

- **Complexity:** Encapsulation can introduce complexity in the design of a system, especially if overused or used inappropriately. Not everything needs to be encapsulated.
- **Overhead:** Introducing getters and setters (methods) for attributes instead of accessing them directly can introduce a tiny amount of overhead. Modern compilers can optimize this overhead, but it's still a theoretical disadvantage.
- **Development Time:** Initially, writing encapsulated code can be slower since you create interfaces for everything and ensure restricted access. However, this often pays off in the long run with easier maintenance and fewer errors.

- **Potential Misuse:** Encapsulation might give a false sense of security if not used properly. For instance, exposing a mutable object via a property might give an impression of safety when, in reality, the object's internal state can still be changed inadvertently.
- **Accessibility Trade-offs:** Sometimes, for encapsulation, developers might make certain components private that might need to be extended or accessed later, requiring refactoring.

When to use the Encapsulation Principle in C#?

Encapsulation is a core principle of object-oriented programming, and its judicious use can greatly improve your software's design, maintainability, and robustness. Here are some common scenarios and use cases where you should consider applying the encapsulation principle in C#:

- **When Protecting Data Integrity:** If an object's internal state needs to be protected from invalid or inconsistent values, encapsulation allows you to enforce validation logic in setters or through methods.
- **To Hide Implementation Details:** When you want the freedom to change the internal workings of a class without affecting external users of the class, encapsulation can hide these details behind a stable interface.
- **When Creating Reusable Components:** When building libraries or components to be used across different parts of an application or even by other applications, encapsulation ensures users of these components interact with them in the intended way without depending on their internal mechanisms.
- **When Managing Dependencies:** Encapsulation can help isolate different system parts so changes in one component don't cascade through the entire application. It aids in achieving a low degree of coupling between software components.
- **To Enforce Usage Patterns:** If there's a specific sequence or pattern in which operations should be performed on an object, encapsulation can help ensure these patterns are adhered to.
- **In Concurrency Scenarios:** When working with multi-threaded applications, encapsulation can help in protecting shared resources from concurrent access issues, for example, by encapsulating locking mechanisms inside the object.
- **To Achieve Abstraction:** In cases where you want to represent a complex system with a more simplified interface, encapsulation can abstract the underlying complexity.
- **When Grouping Related Data and Behavior:** Encapsulation allows related data and the operations that act on them to be bundled, making the code more organized and easier to understand.
- **To Create Custom Types with Specific Behavior:** For example, if you're creating a custom data structure or a domain-specific type, encapsulation can ensure that the behavior and structure of this type are always consistent.
- **For Improved Testability:** Encapsulated components are easier to mock and test in isolation, as their interactions are defined through clear interfaces.

However, it's essential to find a balance. Over-encapsulation can lead to less readable code, where every data attribute and operation is locked away behind accessors and mutators without a clear benefit. As a rule of thumb, you should encapsulate when there's a clear benefit regarding data integrity, abstraction, or maintainability rather than doing it for every class attribute or method.

In the next article, I will discuss the [Multiple Real-time Examples of the Abstraction Principle in C#](#). In this article, I explain **Real-Time Examples of the Encapsulation Principle in C#**. I hope you enjoy this Encapsulation Principle in Real-Time Examples using the C# article.